

Three-Tier Web Application using Flask, MySQL, and AWS

This document contains all screenshots and brief explanations for each phase of the 3-Tier AWS Web Application project.

Phase 1 — Project Planning

Initial architecture design, technology selection, and project setup planning.

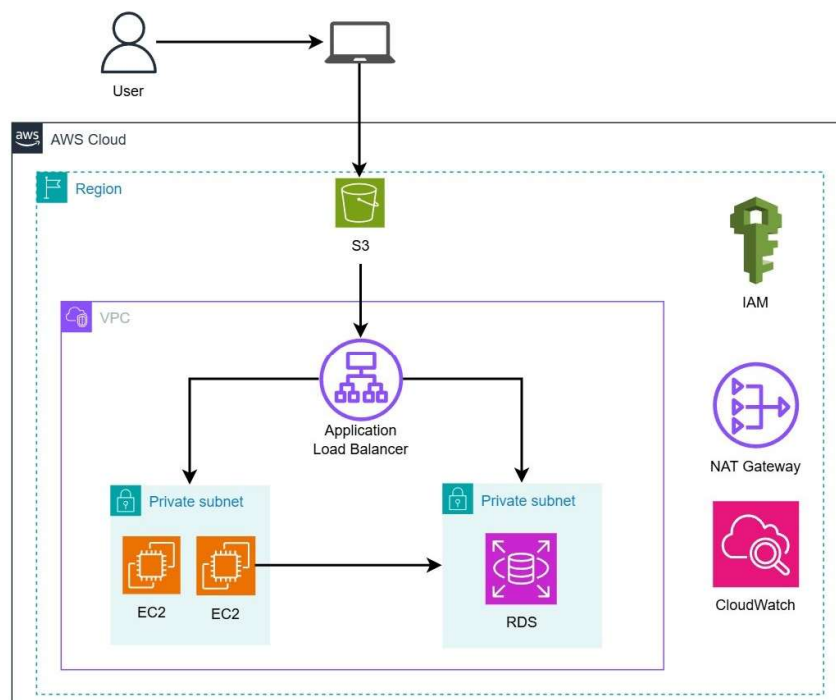


Fig. 1.1. High-level AWS 3-Tier architecture outlining for the Student Management Web Application.

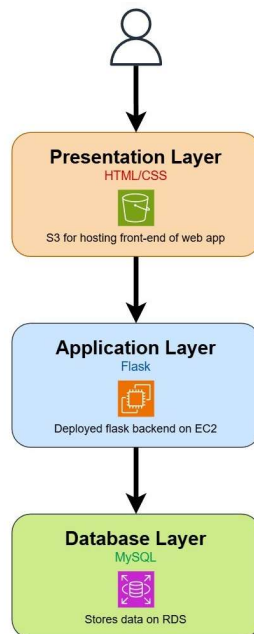


Fig. 1.2. Flow Diagram describing flow of the Web App from Presentation Layer, Application Layer to Database Layer.

🔧 Phase 2 — Database Layer (Amazon RDS)

Setup of Amazon RDS MySQL instance, security group configuration, and database connectivity testing.

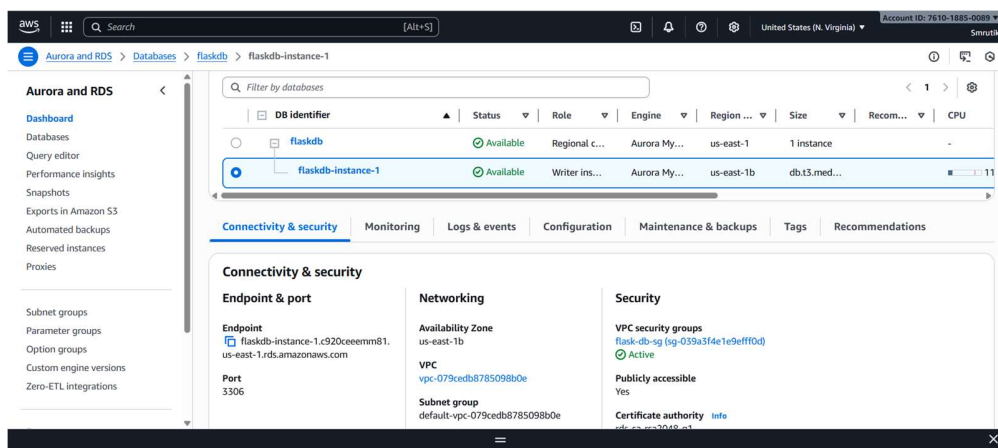


Fig. 2.1. MySQL RDS instance created to store student data with multi-AZ disabled for cost optimization.

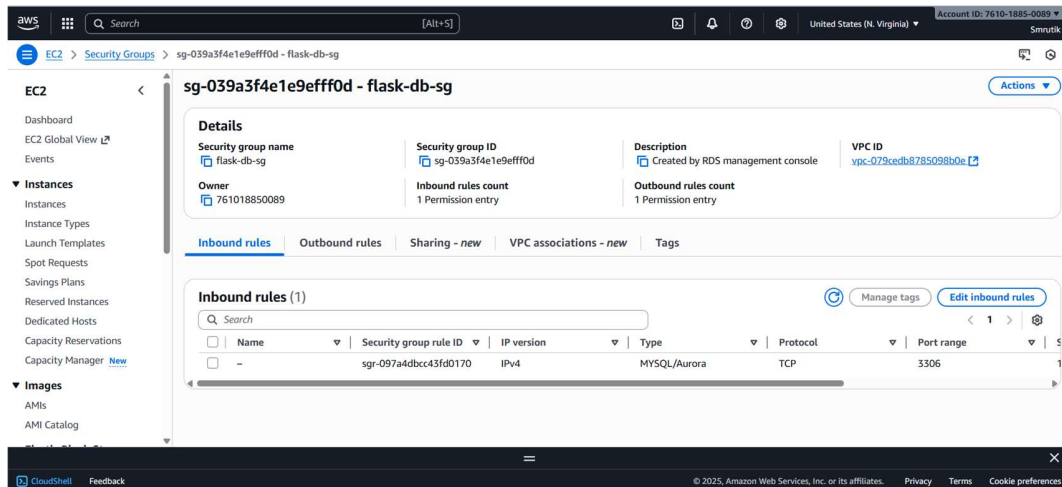


Fig. 2.2. The security group attached to MySQL RDS instance.

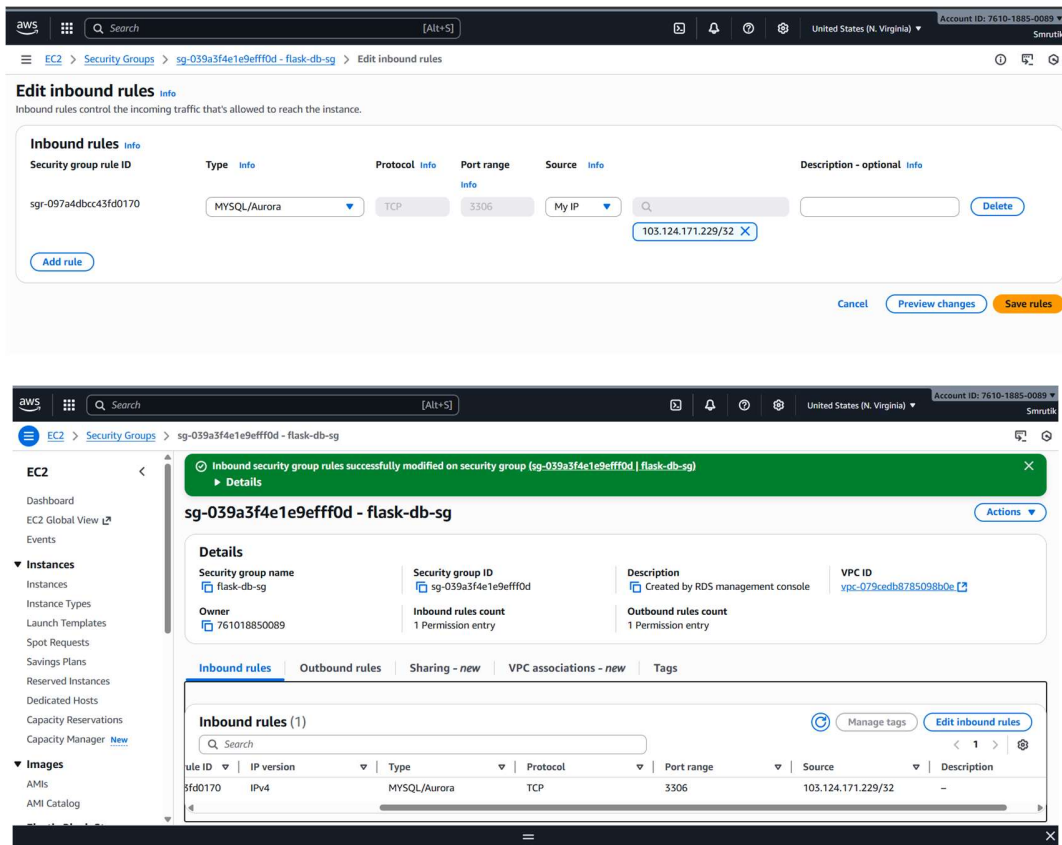


Fig. 2.3. Inbound rules configured to allow security group a single IP address and its connectivity on port 3306.

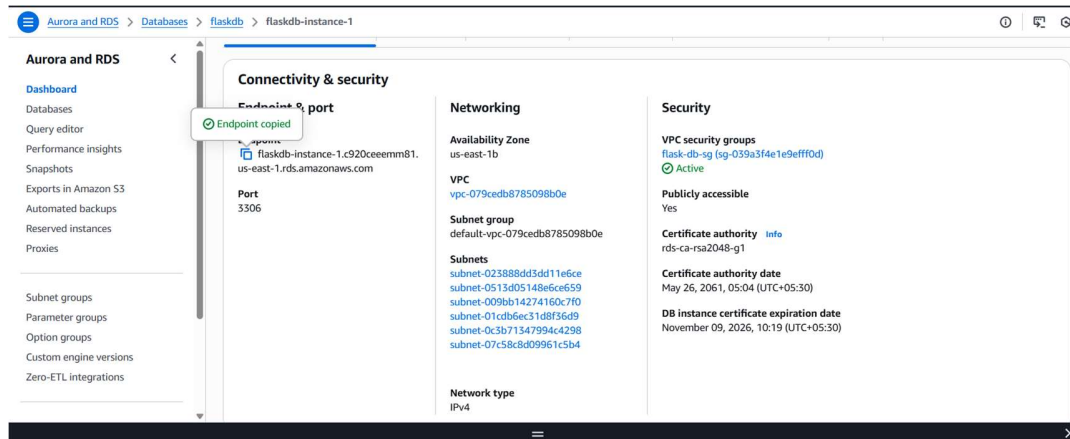
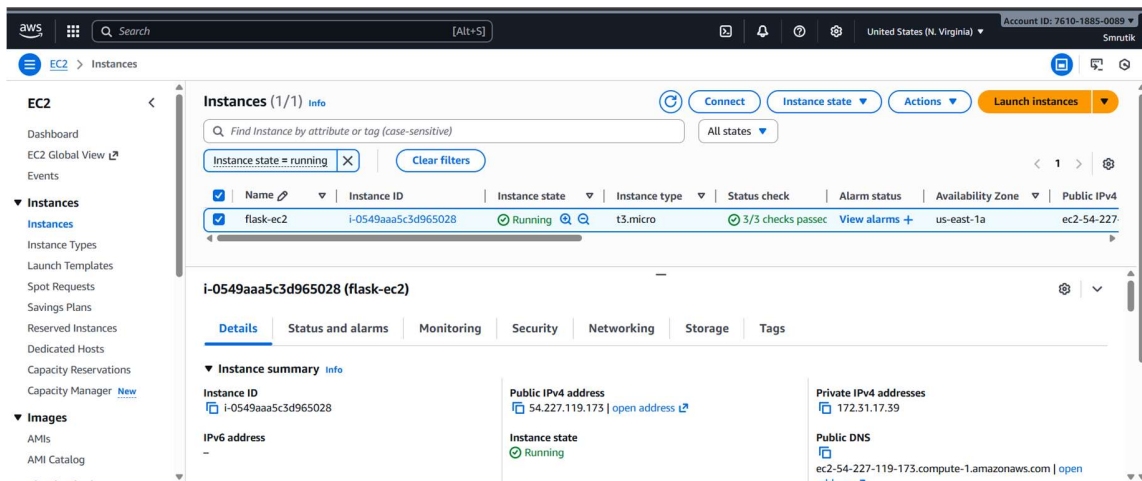


Fig. 2.4. RDS instance active and available, confirming successful deployment of the database layer.

🔗 Phase 3 — Application Layer (Flask on EC2)

Deployment of Flask backend on EC2 instance, installation of dependencies, and backend testing.



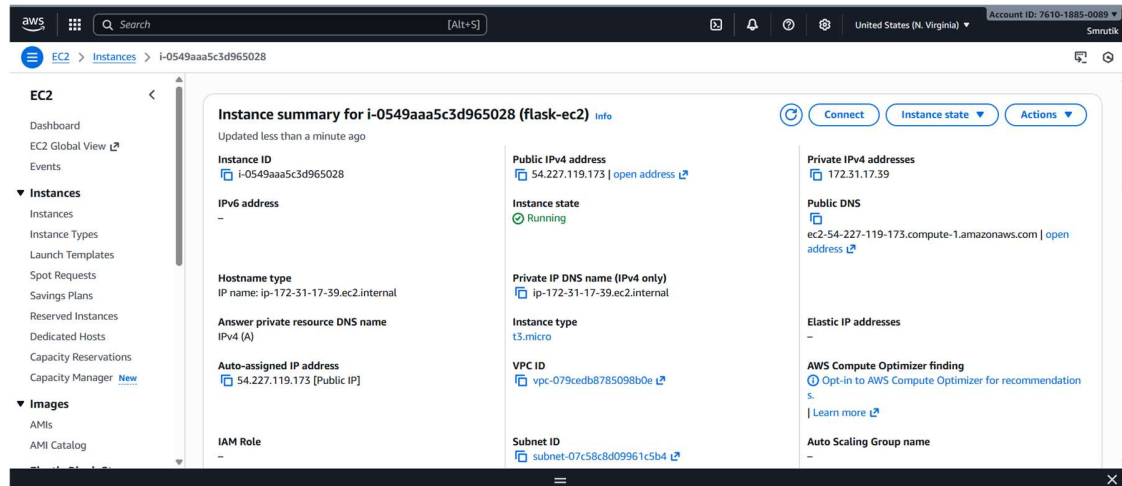


Fig. 3.1. Ubuntu EC2 instance launched to host the Flask backend application, configured with necessary security groups for HTTP and SSH access.

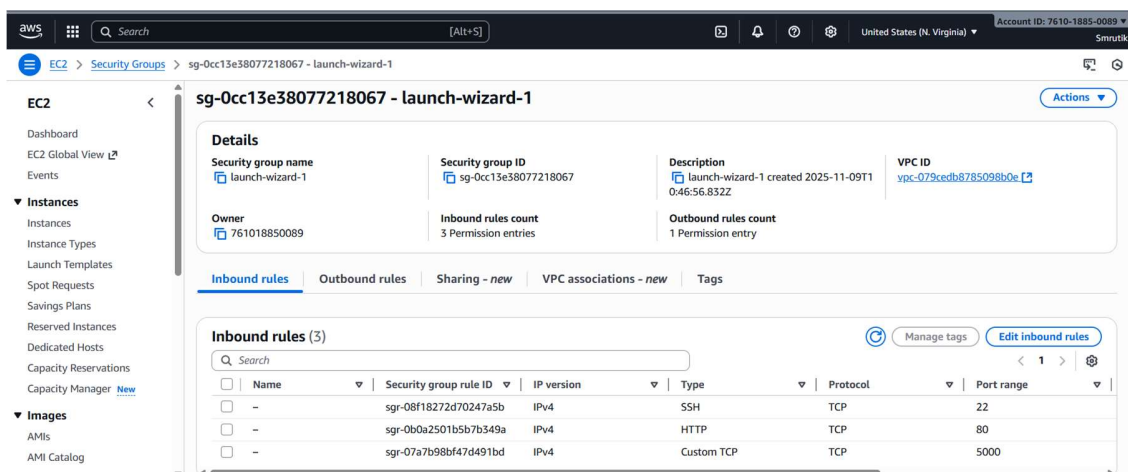


Fig. 3.2. Inbound rules allowing only HTTP (port 80) and application traffic (port 5000) for safe public accessibility while maintaining minimal exposure.

Phase 4 — Presentation Layer (Amazon S3)

Hosting frontend (HTML/CSS) on Amazon S3 and linking it to Flask backend running on EC2.

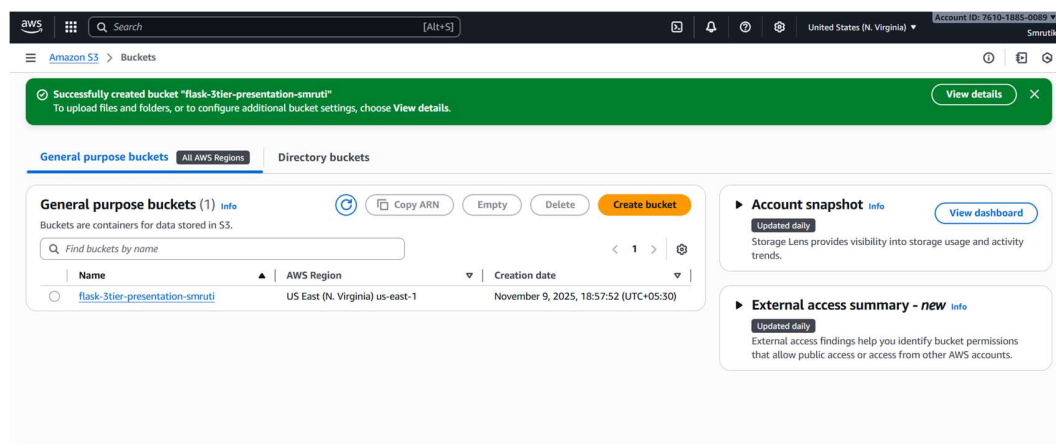


Fig. 4.1. S3 bucket created for static website hosting, acting as the presentation layer of the application.

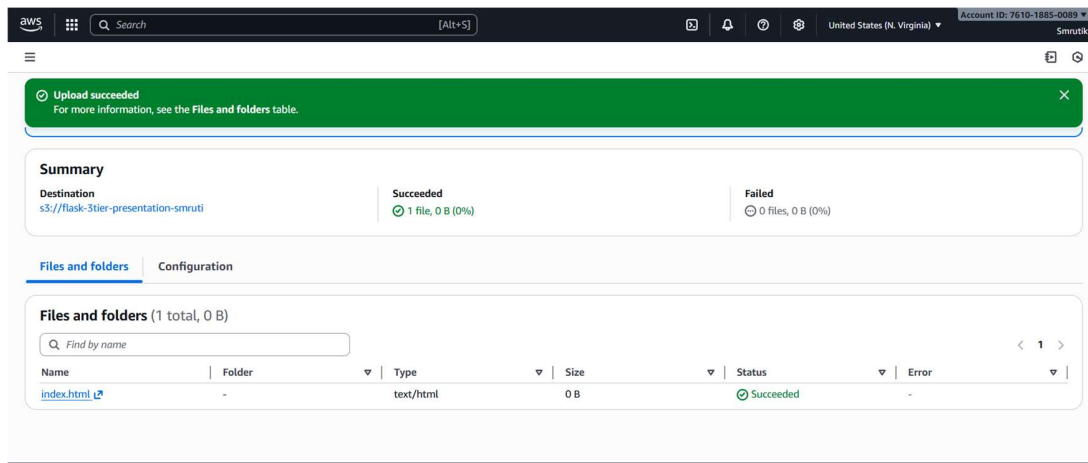


Fig. 4.2. S3 website hosting enabled with an index document defined, making the frontend accessible through a public endpoint.

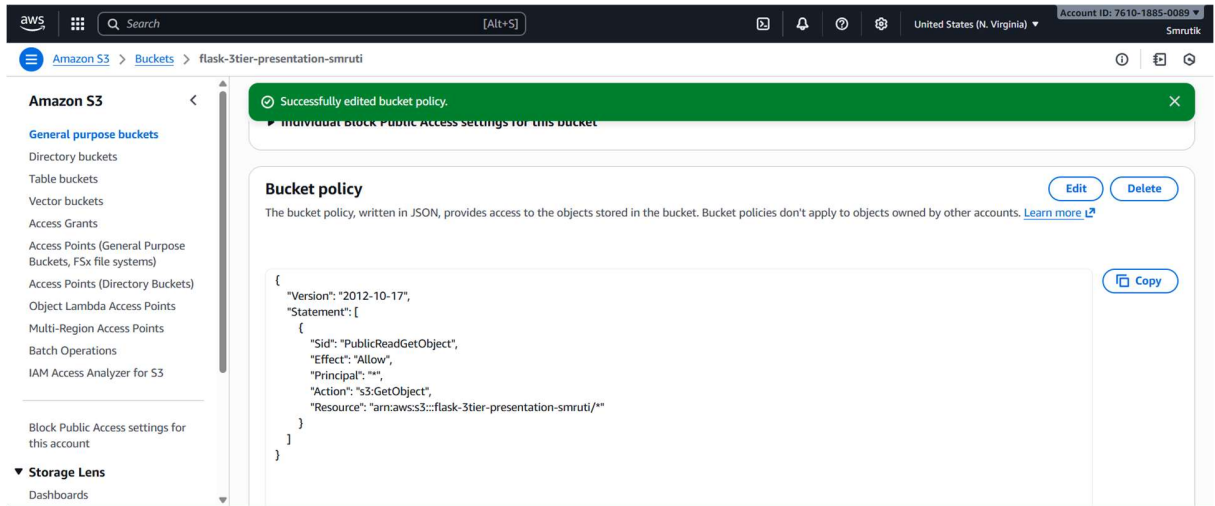


Fig. 4.3. Custom bucket policy applied to allow controlled public read access while ensuring restricted object-level permissions.

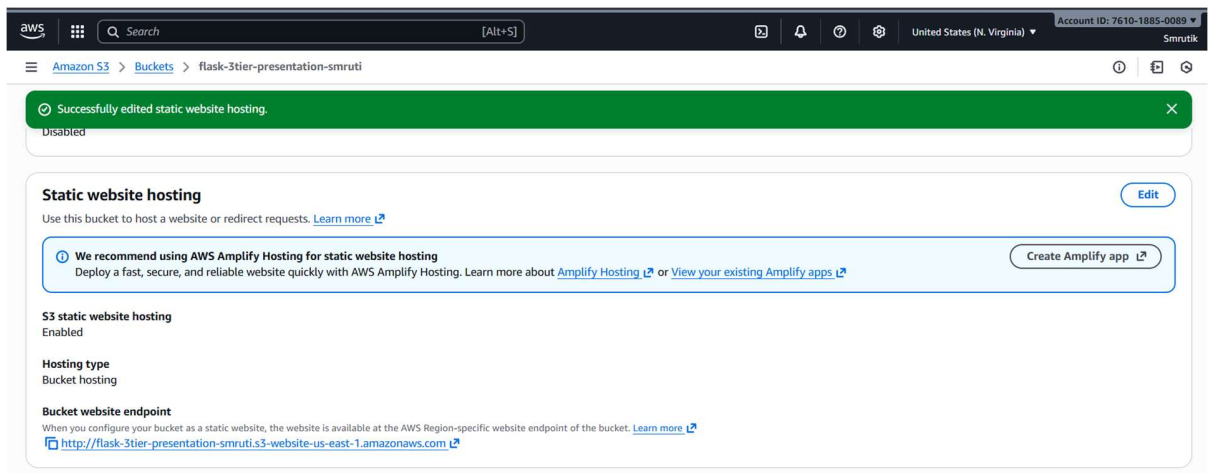


Fig. 4.4. Bucket website endpoint available where front-end interface of Student Management is hosted after successful configuration of S3.

✓ Phase 5 — Final Output

Full system testing ensuring data flow between frontend, backend, and database.

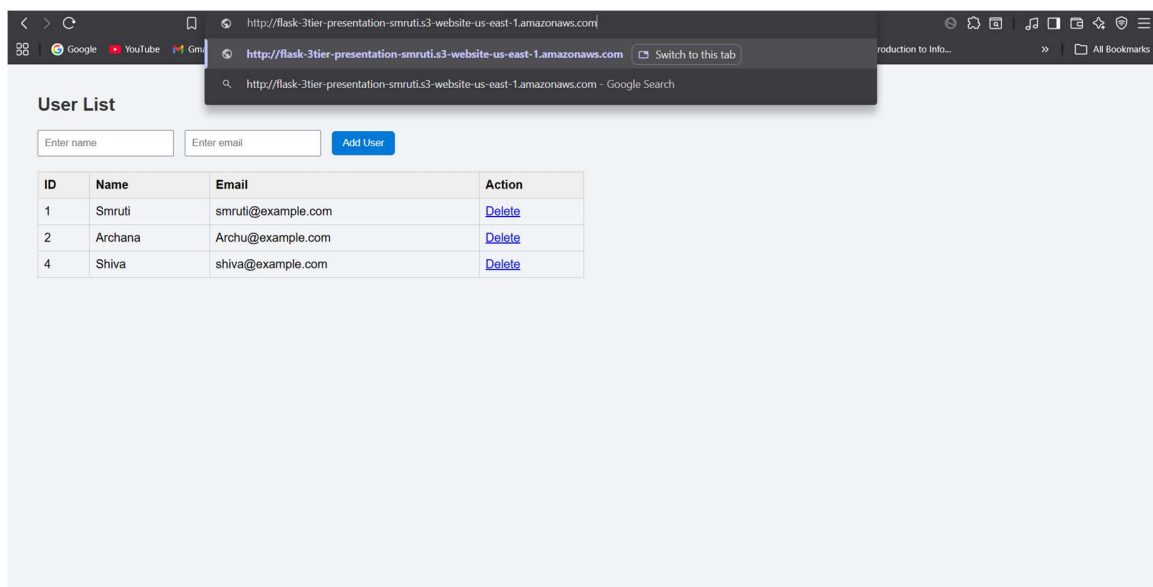


Fig.5.1. Fully integrated web application displaying all student records fetched dynamically from RDS via Flask API.